IMPERIAL

DESE71003 – SENSING AND INTERNET OF THINGS

IMPERIAL COLLEGE LONDON

DYSON SCHOOL OF DESIGN ENGINEERING

Smart Clothing Advisor

Author:
Yu Mi (CID: 06056007)

Code & Data URL:
e.g., https://github.com/myriddle1101-boop/Smart-Clothing-Advisor_Yu-Mi-CID-06056007

Coursework Video URL:
e.g., https://youtu.be/USV_8SZ_MFg

Date: Dec 7, 2025

Smart Clothing Advisor:

A Weather-Aware IoT Clothing Decision System

Abstract-London's unpredictable weather often forces people to rely on intuition and experience when deciding what to wear, rather than on precise environmental awareness. To address this challenge, I designed a user-centered Internet of Things system, Smart Clothing Advisor, that can automatically detect the status of garments and real-time weather conditions. By integrating computer vision, environmental sensing, and actuable physical feedback technologies, the system is capable of providing immediate and tangible outfit recommendations as users dress. Experimental results indicate that, although the system still has limitations in garment-level recognition, its potential to assist users in making more appropriate clothing decisions under varying weather conditions is highly significant.

I. INTRODUCTION

The weather in the United Kingdom is highly unpredictable, making it difficult for people to choose appropriate clothing. Wearing insufficiently or excessively may lead to discomfort, illnesses, or reduced daily performance. People often make decisions based on subjective judgment or incomplete weather information, which has led to the demand for intelligent decision support systems. Since the project is conducted during winter, the main discussion focuses on the warmth of clothing rather than overheating issues. The system captures users' attire through an ESP32-CAM module, uses a customized YOLOv8 model for clothing recognition, retrieves real-time weather and "feels-like" temperature data from the OpenWeather API, and determines the appropriateness of clothing through a warmth score comparison. Subsequently, tactile feedback is provided through different gestures driven by a servo motor. Historical data is stored in a dashboard for long-term pattern analysis.

Objective	Indicators
Detect clothing type using on-device image capture	YOLO-based multi-class detection accuracy Image quality from ESP32-CAM
Retrieve accurate real-time weather and perceived temperature	API response time Temperature & feels-like accuracy
Compare clothing warmth score with current weather to infer comfort level	Clothing warmth scoring model design Thresholds for "Too Cold / Warm Enough" classification
Provide physical feedback using servo-based actuation	Response latency (image → decision → servo) Accuracy of gesture output (thumbs-up /down) System stability over repeated cycles

TABLE 1: Project Key Objectives

II. SYSTEM OVERVIEW

The system comprises four main modules:

Sensing Layer	ESP32-CAM captures JPEG images when the button on the webpage is pressed， Weather API supplies temperature and thermal comfort metrics.
Edge Processing Layer	A Flask backend receives images and performs YOLOv8 inference. Clothing categories such as thick inner/thin inner/ coat/ down jacket are identified. Each garment class is mapped to a warmth score.
Decision Logic Layer	Using total warm-score and real-time "feels-like" temperature, a rule-based model computes "cold / suitable".
Actuation & Application Layer	ESP32 with servo motor produces corresponding gesture. A web dashboard visualizes historical temperature, clothing choices, and comfort outcomes.

TABLE 2: System Overall Architecture

III. SENSING

A. Sensing Scenario

When users need to choose clothing, they simply stand in front of the ESP32-CAM device next to the bedroom wardrobe. After putting on clothes and remaining still, they lightly tap the capture button on the mobile web page, and the device immediately takes a snapshot and identifies the upper-body clothing layer. This creates a natural and convenient sense–decide–actuate workflow, enabling daily usage with minimal friction.

B. Heterogeneous Time-Series Data Sources and Acquisition Setup

The system uses 3 distinct time-series data types:

Data Type	Source	Sampling Rate	Purpose
Visual sensing	ESP32-CAM	event-driven(Press button)	Detect outfit layers
Environmental sensing	OpenWeather API	event-driven	Obtain temperature & feels-like
System logs	Flask server	event-	Time-stamped

		driven	recognition results

TABLE 3: Data Types

Instead of streaming continuous video, the system operates in an event-driven(press the button) capture mode to reduce bandwidth usage and minimize the computational load.

After dressing, the user opens the mobile webpage on the phone and taps the "Take a photo" button. This manual trigger sends a request to the ESP32-CAM to take a single photo of the user's upper-body clothing. The captured image is immediately transmitted to the local flask server, where the YOLO-based model processes it to identify clothing categories. The mechanism ensures that data is collected only when needed and aligns with natural behaviour patterns, making the system more reliable and user-controlled.

Unlike the camera which is triggered by the ESP32 Cam's web server, the OpenWeather API is queried by the Flask backend immediately after the image upload is acknowledged and the YOLO inference thread is initiated. The system uses the API to fetch local weather conditions, specifically the feels_like temperature. This metric is a superior predictor of human thermal comfort than the ambient temperature, as it factors in wind chill and humidity.

C. Data Communications & Networking

The data communication architecture is event-driven and asynchronous, designed to provide immediate feedback to the ESP32-CAM while offloading heavy computation to the server.

Stage	Protocol / Direction	Key Action
Trigger	ESP32-CAM to User (Mobile)	The user opens the mobile browser to the ESP32's IP and manually initiates the capture.
Upload	ESP32-CAM to Python Server	The ESP32-CAM sends the raw JPEG byte stream via HTTP POST /upload. The server immediately returns a 200 OK acknowledgment, allowing the ESP32 to close the connection quickly, thus decoupling the upload latency from the YOLO inference time.
Inference	Flask Server (Main Thread)	Immediately acknowledges upload (200 OK) and delegates ML task to a background thread.
Data Fusion & Decision	Flask Server (Background Thread)	YOLO inference, OpenWeather API query, and final comfort scoring calculation.
Actuation	Python Server (Client) → Servo Control	Server sends the calculated status code to the dedicated servo control board.
Publishing	Flask Server (API) → Dashboard	Server updates the JSON Manifest; Dashboard dynamically queries the data.

TABLE 4: Data Communication

In term of Reliability and Encoding, communication uses TCP/IP over 2.4 GHz Wi-Fi. Image data is encoded as binary JPEG, and control/log messages use JSON or URL parameters. To ensure the consistent performance, camera parameters like exposure and gain were manually tuned on the ESP32 to stabilize image quality and transmission size. These improvements resulted in more consistent latency and fewer corrupted uploads.

D. Storage, Publishing & Dataset Management

Smart Clothing Advisor stores all the data into distinct components on the local server, structured for easy retrieval and auditing.

1. Image Files

For raw images, The original JPEG images are saved within a structured folder hierarchy. Annotated images with YOLO bounding boxes are saved alongside the raw images to support immediate visual feedback on the web interface.

2. Central Structured Dataset

The entire system history is stored in a central JSON Manifest. This file records paths to the image files, detected clothing labels, real-time weather snapshots, computed warmth scores, and the final decision status. Therefore, user can see the entire historical record on the local server.

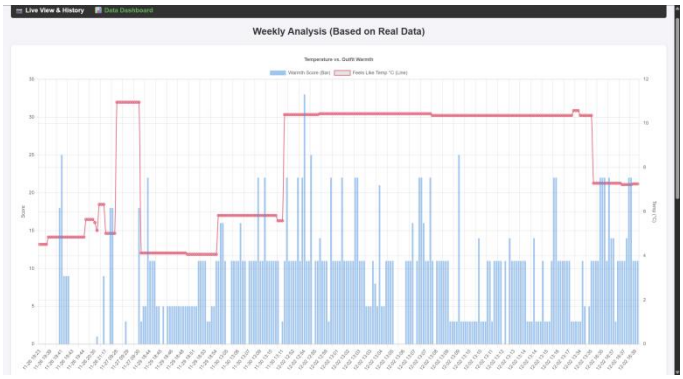
3. Real-time Environmental Data Acquisition

Environmental data is acquired on-demand by the Python backend. Specifically, the OpenWeather API is queried immediately following the successful upload and initial processing of a new image (within the background processing thread). This ensures that the environmental context is the closest possible real-time snapshot relative to the captured outfit.Additionally, it reduces times of queries for API.

E. Basic Time-Series Data Analysis

All analysis and aggregation are performed dynamically by the Python backend's /api/stats endpoint, ensuring that visualizations are always reflective of the latest data.

The resulting data stream consists of three synchronized elements: YOLO calculated warmth scores, real-time feels_like temperature data, and the system's final two-state comfort classifications. This dynamic analysis framework confirms the system's potential to extract meaningful, real-time behavioral patterns from heterogeneous IoT data.



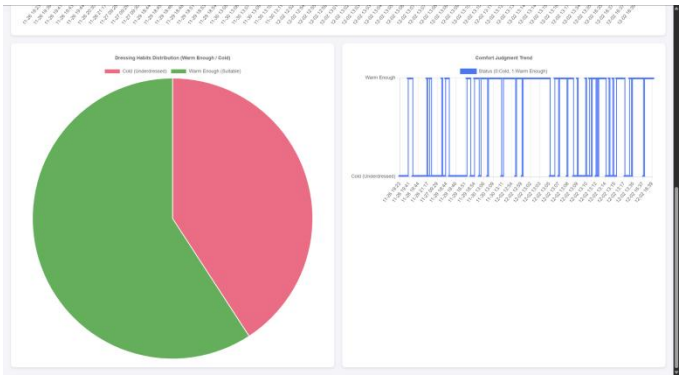


Figure 1: Dashboard on webpage

IV. IoT PROTOTYPE

A. Machine Learning: YOLOv8 for Clothing Recognition

The core intelligence of the system is based on an object detection model, YOLOv8, trained to identify specific clothing items of upper body. Considering the layering of clothes in winter, the original model should have been able to recognize multiple layers, but after testing, this feature was ultimately abandoned in light of bad performance.

1. GPU Acceleration

The computer vision subsystem utilizes the YOLOv8 architecture, which is selected for its optimal balance between inference speed and detection accuracy. Training was accelerated by using a high-performance NVIDIA GPU. Leveraging CUDA cores significantly reduced the training time per epoch, allowing for rapid hyperparameter tuning and iterative model refinement compared to CPU-based training.

It is worth mentioning that during the development process, a significant portion of effort was spent setting up a functional GPU-accelerated environment for running YOLO and PyTorch. The process involved multiple compatibility issues related to GPU architecture, CUDA versions, PyTorch builds, and Python package conflicts. My laptop uses the new NVIDIA RTX 5060 (sm_120), which was not supported by the first PyTorch version I installed, causing the GPU to remain unavailable despite having a correct CUDA driver. I initially suspected a CUDA mismatch, but later learned that PyTorch must explicitly support the GPU architecture. Therefore, I upgraded it to a newer PyTorch release, and finally resolved the issue. Several dependency problems also appeared along the way: outdated pip versions prevented packages from installing, PyQt build failures blocked LabelImg, and ultralytics required a newer Python–NumPy combination. Ultimately, I recreated a clean Conda environment using Python 3.9 and reinstalled all libraries, then aligned the PyCharm interpreter with the new environment. After that, the tool chain became stable. Although these challenges were time-consuming, they helped me understand GPU compatibility, CUDA–driver relationships, and Python environment management.

2. Data Preparation and Iterative Refinement

The dataset construction involved a critical iterative process to optimise system performance. All images were manually annotated using LabelImg, generating standard YOLO-format .txt files containing bounding box coordinates and class indices.

When I began to annotate my dataset with LabelImg, I repeatedly encountered errors caused by PyQt6, such as “argument has unexpected type ‘float’,” which prevented the canvas and drawing tools from working properly. The core reason turned out to be that LabelImg is only compatible with PyQt5, but my system environment was automatically loading PyQt6. After several trials, I resolved the problem by completely uninstalling all PyQt6-related packages, reinstalling PyQt5, and then installing a PyQt5-compatible version of LabelImg.

Initially, the model was trained on an 8-class schema including sweater, hoodie, coat, cotton_jacket, t_shirt, skirt, trousers, and thin_jacket. However, early validation results were unsatisfactory. The model struggled with high inter-class confusion. Also, since the ESP32 cam can't capture the lower-body clothes, it wasted computational resources detecting lower-body garments. I didn't have that many samples for classifying clothes with so many labels, and it is unnecessary. Based on these findings, the dataset was redesigned into four distinct and functionally relevant thermal categories: thick_inner, thin_inner, down_jacket, coat. This simplification improved model convergence and directly aligned the classification output with the logic required for the warmth score algorithm.

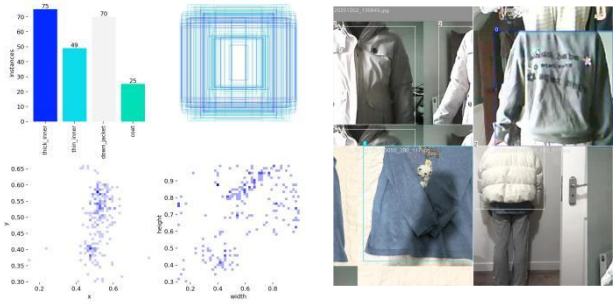


Figure 2: Labels and Train_batch

3. Training Analysis & Visualisation

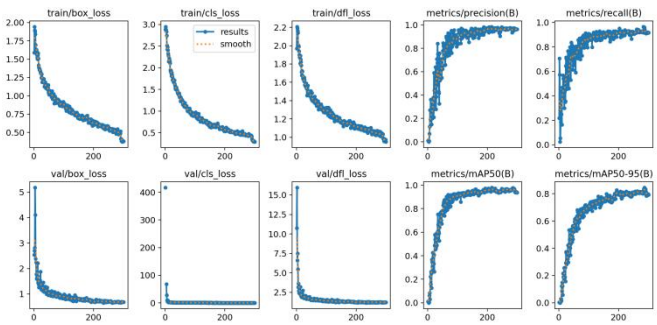


Figure 3: Training and Validation Loss Curves

The final model was trained for 300 epochs with a batch size of 4(used to be 16 but reduced because insufficient running memory). The training process generated several key visualizations that validate the model's performance.

As shown in Figure 3, both the Box Loss (localization error) and Class Loss (categorization error) show a consistent downward trend, stabilizing after approximately 150 epochs. The convergence of training and validation loss indicates that the model learned the features effectively without suffering from significant overfitting.

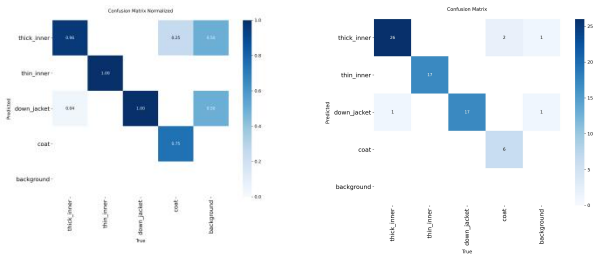


Figure 4: Confusion Matrix on the Test Set

The confusion matrix(Figure 4)reveals high precision for distinct categories. Notably, thin_inner and down_jacket achieved a perfect recall rate (1.00), demonstrating its exceptional ability to identify all instances within these categories.The model has some difficulties in distinguishing between thick inner and down jacket /coat. The biggest issue is the background category. The original matrix shows that the model misclassified 3 background samples as other categories. I originally thought that since this model was ultimately just for my own use, I only needed to use my own clothing samples, so there were fewer samples in the dataset. However, the results showed that although the accuracy seemed high, the final recognition results were not as good as shown in the figures.

The F1 curve(Figure 5) demonstrates an optimal threshold around 0.31, achieving a high overall F1 of approximately 0.95, supporting its suitability for real-time deployment on the IoT pipeline. The F1 curve for the coat category appears slightly lower than other categories, particularly in the higher confidence region, which may indicate that detection of this category is slightly more challenging.

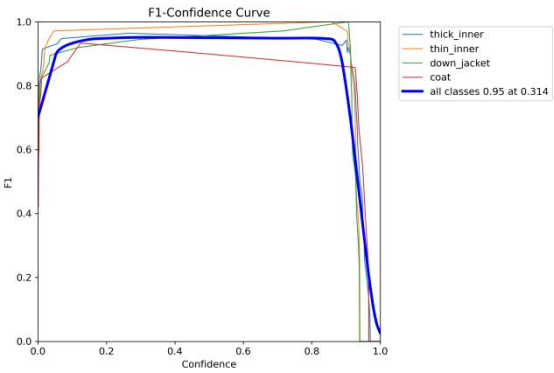


Figure 5: F1-Confidence Curve

4. Limitations & Critical Analysis

Apart from the issues caused by the small sample size mentioned above, this model also has some inherent limitations, especially regarding Multi-Layer Occlusion. The current system struggles to identify inner layers when they are fully or mostly covered by an outer layer. The system tends to detect only the outermost visible garment. Although I intentionally included quite a few samples in the dataset where jackets were open to reveal the inner clothing and used multi-label classification approach, the training results were not satisfactory. I believe this is because when using Labelimg to draw boxes, there were cases of overlapping labels. While theoretically feasible, this may require a massive, high-quality sample dataset. In addition, YOLO is an optical object detector. It relies entirely on visible pixel features. Unlike X-ray imaging, it cannot infer the existence of an object that is fully occluded. While it can handle partial overlap, it cannot detect a garment that has no visible features in the frame. Future iterations would require a significantly larger dataset featuring "layered" outfits.

B. Warmth Score Mapping & Data Fusion Logic

To quantify the thermal insulation of an outfit, a heuristic scoring system was mapped to the detected classes. This mapping transforms qualitative classification labels into quantitative data suitable for mathematical fusion.

number	Clothing Class	Symbol	Warmth Score
0	thick_inner	sweater / hoodie	5
1	thin_inner	t-shirt / base layer	3
2	down_jacket	puffer jacket/cotton jacket	11
3	coat	trench Coat / wool coat	9

TABLE 5: Warmth Score

```
def is_warm_enough(cloth_name, feels_like_temp):
    base_score = clothes_warmth.get(cloth_name, 0)

    if feels_like_temp < 0:
        required_score = 11
    elif feels_like_temp <= 5:
        required_score = 9
    elif feels_like_temp <= 10:
        required_score = 7
    else:
        required_score = 3

    return base_score >= required_score
```

Figure 6: Decision Logic

In order to determine how warm or cold one's clothing is, the Warmth Score is integrated with environmental context. The system sums the scores of all detected garments in the frame(This is because it was originally designed for multi-label recognition, capable of identifying both inner wear and outerwear at the same time). Then a required warmth threshold is determined based on the current "feel_like" temperature fetched from the API. The warmth threshold is determined by personal clothing habits. For example, when the feel_like temperature is below zero, I have to wear a down jacket to stay warm.

Finally, the system automatically compares the total warmth score with the weather-based warmth threshold to draw a conclusion.

C. IoT Actuation: Servo Control

```
void handleServoCommand() {
  if (server.hasArg("status")) {
    int status = server.arg("status").toInt();

    int targetAngle = centerAngle;

    switch (status) {
      case 1:
        targetAngle = thumbsUpAngle;
        Serial.println("Received: THUMBS UP (Warm Enough)");
        break;
      case 0:
        targetAngle = thumbsDownAngle;
        Serial.println("Received: THUMBS DOWN (Too Cold)");
        break;
      case 2:
      default:
        targetAngle = centerAngle;
        Serial.println("Received: CENTER (Unknown)");
        break;
    }
```

Figure 7: Servo Status Logic

The actuation subsystem provides the final, physical feedback to the user, immediately translating the server's algorithmic decision into a tangible “thumb” performance. This is implemented using a dedicated ESP32 Web Server running the Arduino code provided, which acts as a lightweight actuator controller. The actuation relies on a simple, yet robust, machine-to-machine communication via HTTP. The central Flask server acts as the HTTP Client, pushing the final comfort decision to the Servo ESP32, which acts as the HTTP Server. After the Data Fusion Logic completes, the Flask server sends an asynchronous HTTP GET Request to the Servo ESP32's local IP address. The Servo processes the request instantly and returns a 200 OK response to confirm command reception. This physical actuation provides immediate, non-intrusive feedback to the user without requiring them to check the web dashboard.

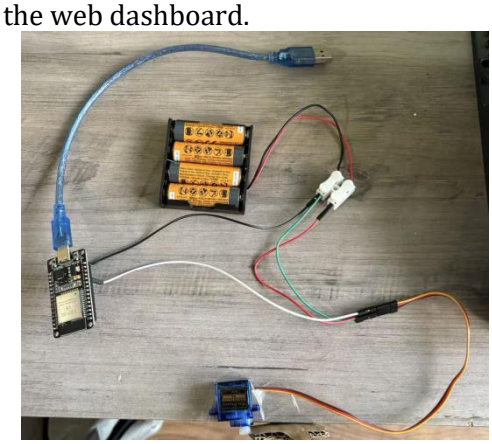


Figure 8: Wiring Diagram

I connected the signal line of the SG90 servo motor to the ESP32 board, and power it separately with a battery, and ensure that the ESP32 and the servo motor share a common ground to guarantee stable operation.

D. Performance Analysis

The system's performance was evaluated based on the core metrics of a typical IoT application: latency, resource efficiency, and scalability.

1.End-to-End Latency

Latency measures the total time from the user pressing the "Capture" button to the Servo Actuator moving. The system uses an asynchronous design to mitigate the impact of slow component. However, experiments have shown that YOLO model inference is not the main bottleneck. The main cause of the delay is image, primarily dependent on the Wi-Fi bandwidth required to transmit the high-resolution JPEG image from the ESP32-CAM. Because the ESP32-CAM has relatively high network requirements (it must be 2.4GHz), this project uses a personal hotspot, which causes a delay in real-time photo updates.Overall, aside from a delay in real-time photo updates on the webpage, the servo motor responds very quickly to signals. The time from when the user requests to take a photo to when the system finally reacts is acceptable.

2.Resource Efficiency

In terms of energy consumption, the system adopts an on-demand (event-driven) capture strategy. The ESP32-CAM remains in a low-power idle state most of the time, only consuming peak current during the brief photo capture and Wi-Fi transmission. This is significantly more energy-efficient than continuous video streaming. At the same time, the command to call the weather API is also event-driven, significantly saving API resources.

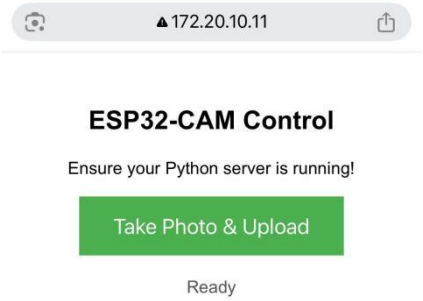


Figure 9: ESP32-CAM control page

V. Limitations & Opportunities

A. Limitations

1. Limitation of model

Obviously, the current project has significant limitations. The recognition accuracy is not high, and I believe there are two reasons for this. Firstly, the model's sample size is insufficient, and the quality varies. Due to constraints in equipment, budget, and time, although the dataset has now been expanded to about 300 images, the similarity in shape between different garments makes clothing recognition difficult. I considered using a clothing recognition dataset (DeepFashion2 Dataset) during training, but because this dataset is too large and the clothing

categories are too detailed, I finally quit it. Secondly, the ESP32-CAM has a low shooting resolution and is highly affected by lighting conditions. To capture the entire upper body, the camera needs to be positioned relatively far away. When the clothing is dark in color, it is difficult to detect the presence of the object.

During model training, there were actually two iterations. In the second iteration, more photos taken with the ESP32-CAM were added to the training dataset, and the model performed slightly better compared to the first version.

2.Limitation of decision logic

In terms of decision logic, since the project background is winter in London and in summer users only need to wear short sleeves, the demand for “not knowing what to wear” is relatively low, so the decision logic only consists of “cold” and “warm enough.” Currently, the model cannot assess layered clothing, so the clothing decision logic is very simple and cannot actually satisfy the 'comparison between the total warmth score and the temperature threshold' requirement (with the current logic, as long as a down jacket is worn in cold weather , the system will not judge that the user is underdressed). Furthermore, due to framing limitations, the final project can only assess the upper body and cannot assist users with clothing choices for the lower body.

B. Future Opportunities

In the future, intelligent dressing assistants have great potential and can be expanded into the clothing matching app market. From a user personalization perspective, different users can set warmth ratings and weather perception thresholds for their clothing during the setup stage. This allows the judgment system to better understand user preferences and make more accurate decisions. For calling weather APIs, the system can access weather conditions worldwide to help users prepare clothing needed for trips, or use weather forecasts to prepare clothing for the coming week in advance. When it comes to clothing recognition, in addition to enlarging the dataset and addressing current limitations, the system can not only be limited to simply recognizing clothing categories, but also identify colors, styles, and other attributes, helping users with advanced clothing matching. For sensor integration, if a temperature and humidity sensor is integrated, the system can help users decide what to wear indoors, further expanding the usage scenarios.

Reference

[1] Jocher, G., Chaurasia, A., & Qiu, J. (2023). YOLOv8 by Ultralytics. GitHub. https://github.com/ultralytics/ultralytics
[2] Complete Guide to Installing PyTorch CUDA on RTX 5070 Ti: Fixing SM_120 Compatibility Issues-Arxu-Blog Garden (2025). Cnblogs.com. https://www.cnblogs.com/arwen-xu/p/19162267
[3] PyTorch. (n.d.). Www.pytorch.org. https://pytorch.org/get-started/previous-versions/
[4] CUDA Toolkit Archive. (2013, July 30). NVIDIA Developer. https://developer.nvidia.com/cuda-toolkit-archive
[5] xiaoyanboke. (August 4, 2025). A detailed guide to installing PyTorch with CUDA on RTX 5060 graphics cards. Csdn.net. https://blog.csdn.net/xiaoyanboke/article/details/149910509
[6] Build software better, together. (2025). GitHub. https://github.com/search?q=ESP32+Camera+Web+Server+Example&type=issues
[7] DeepFashion2: A Versatile Benchmark for Detection, Pose Estimation, Segmentation and Re-Identification of Clothing Images, arXiv preprint arXiv:1901.07973, 2019.
[8] Sapkota R, Karkee M. Ultralytics YOLO evolution: An overview of YOLO26, YOLO11, YOLOv8 and YOLOv5 object detectors for computer vision and pattern recognition[J]. arXiv preprint arXiv:2510.09653, 2025.
[9] Chang Y H, Wu F C, Lin H W. Design and implementation of esp32-based edge computing for object detection[J]. Sensors, 2025, 25(6): 1656.
[10] Bhavana B R, Maruthi H N, Mallikarjun H S. Smart Wardrobe Systems: A Survey of AI, ML, and DL Models for Weather and Mood-Based Clothing Recommendations[J]. Journal of Systems Engineering and Electronics (ISSN NO: 1671-1793), 2025, 35(9).
[11] Monga S, Ranade P, Verma I K. A User-Centric Approach to Revolutionizing Wardrobe Management: Integrating Connected Sensors and AI-Driven Outfit Suggestions[C]//International Conference on Information Technology and Artificial Intelligence. Singapore: Springer Nature Singapore, 2025: 573-586.
[12] OpenWeatherMap.org. (2015). Weather API - OpenWeatherMap. Openweathermap.org. https://openweathermap.org/api

*Statement：Some text in the report has been AI-optimized.

Appendix

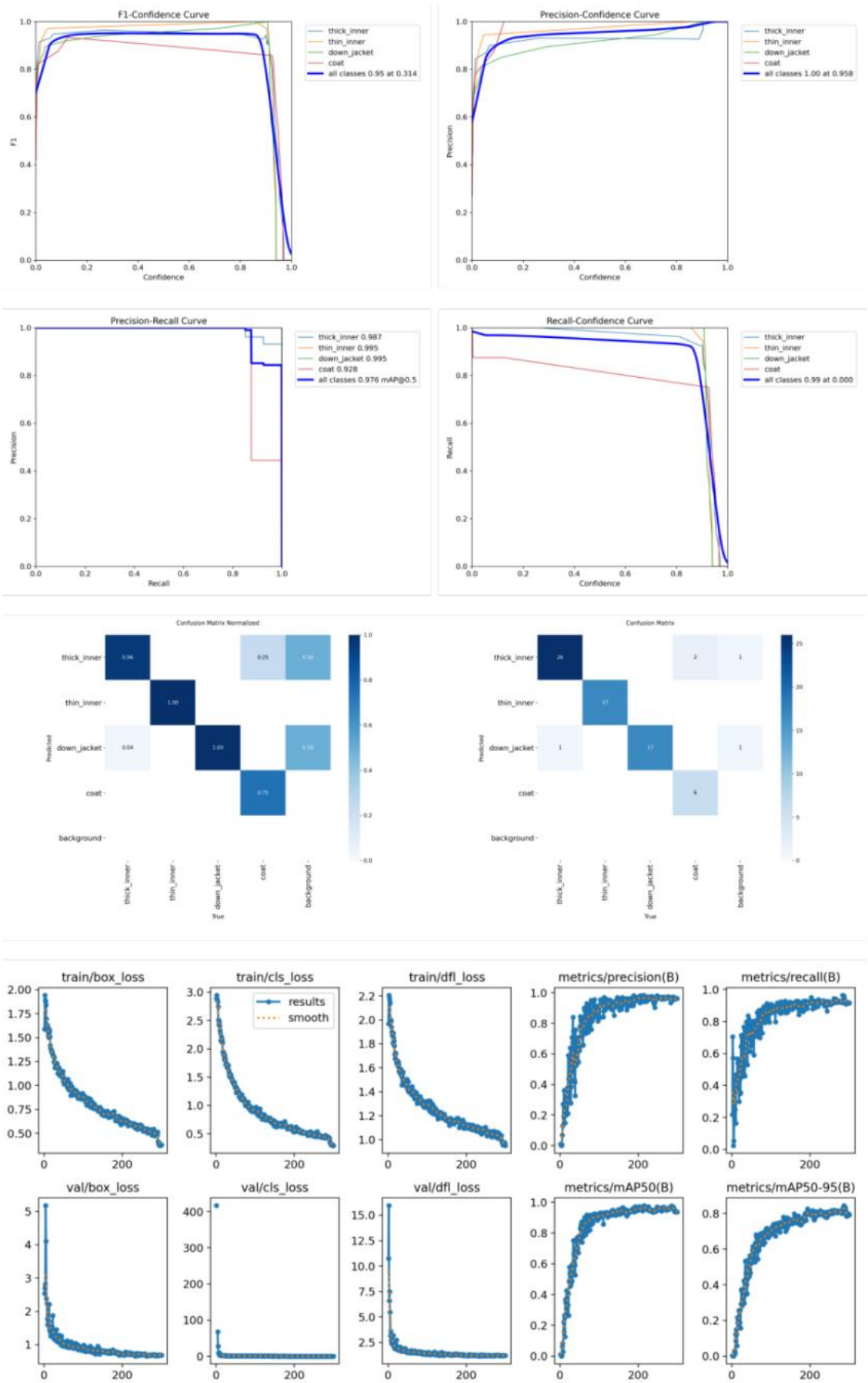


Figure1: result for final model

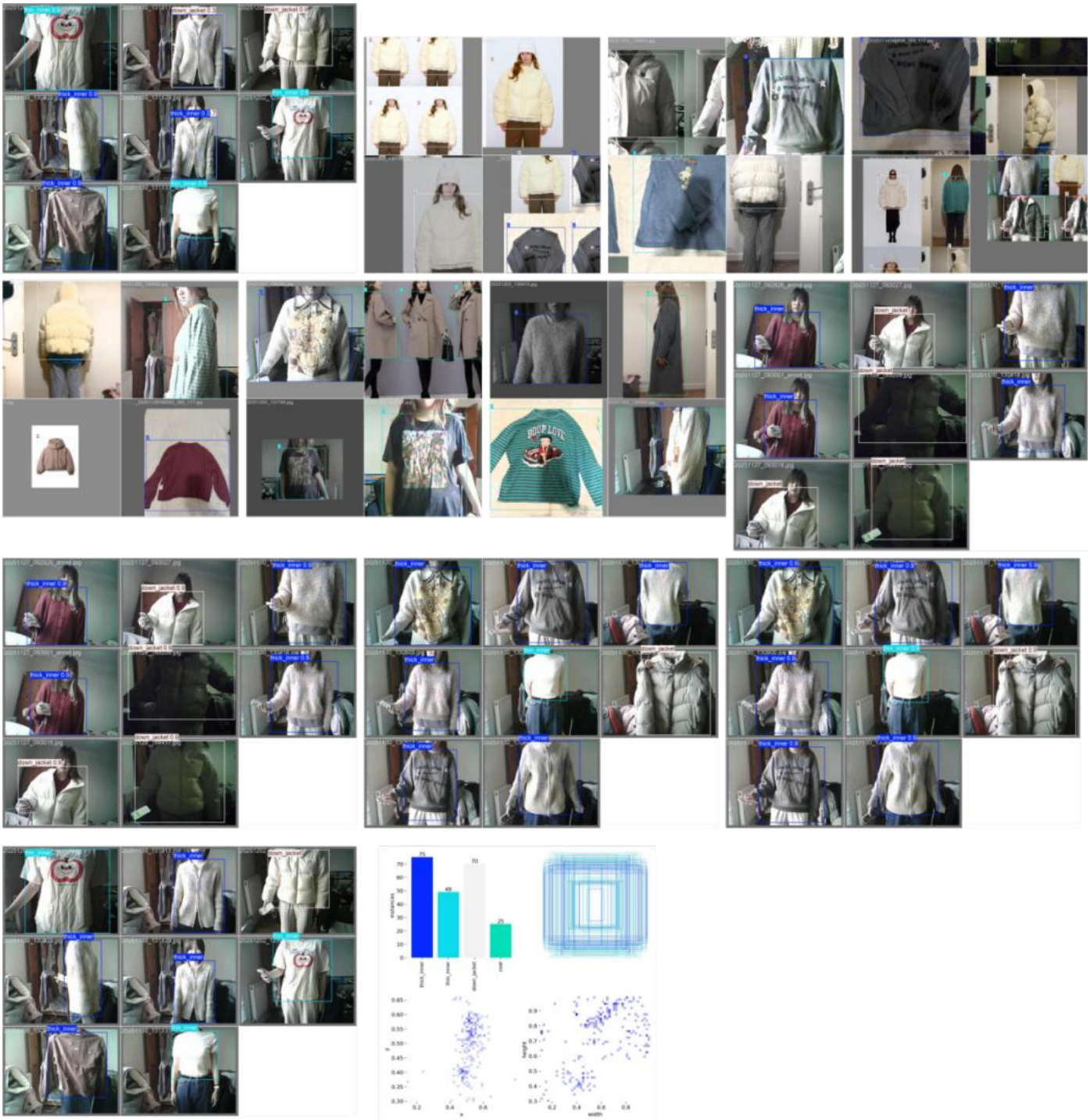


Figure2: train/val_batch for final model

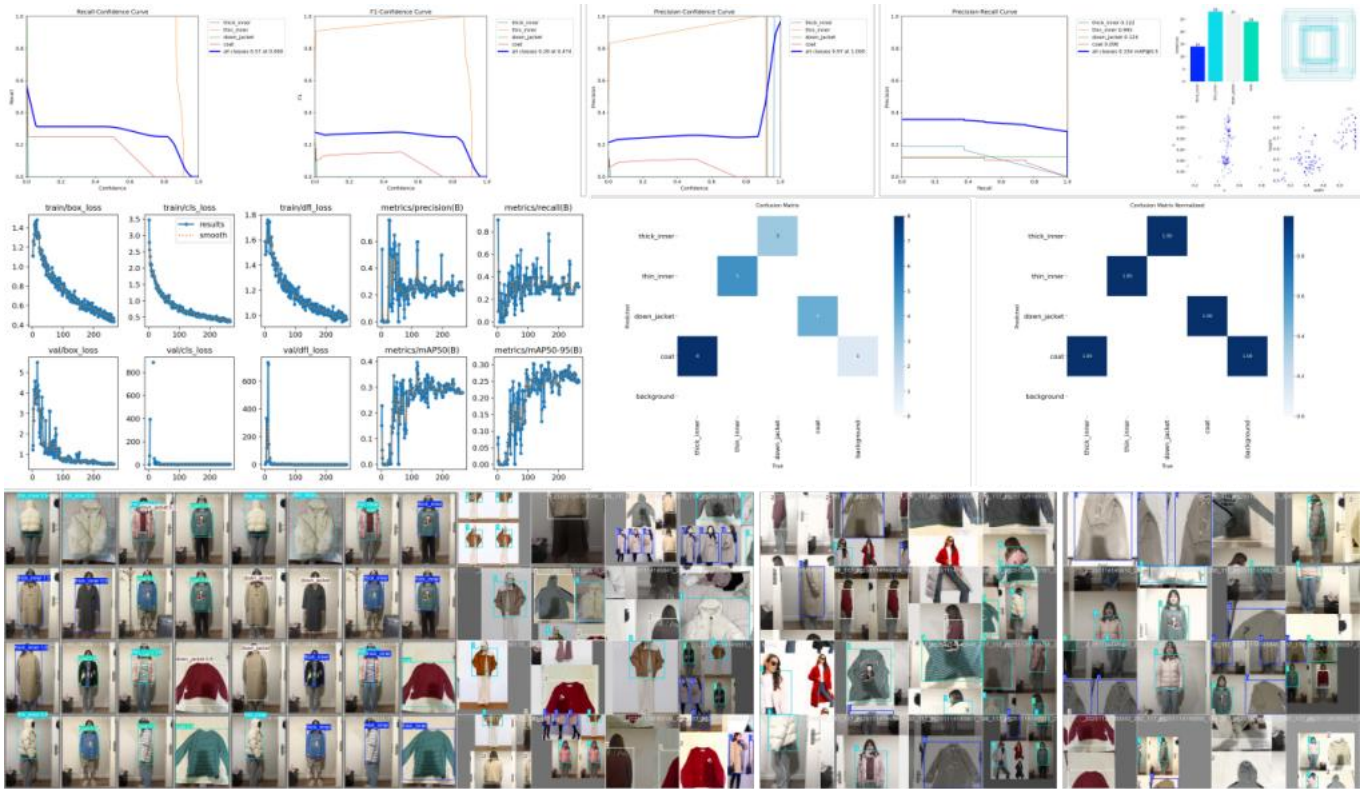


Figure3: result for previous model1

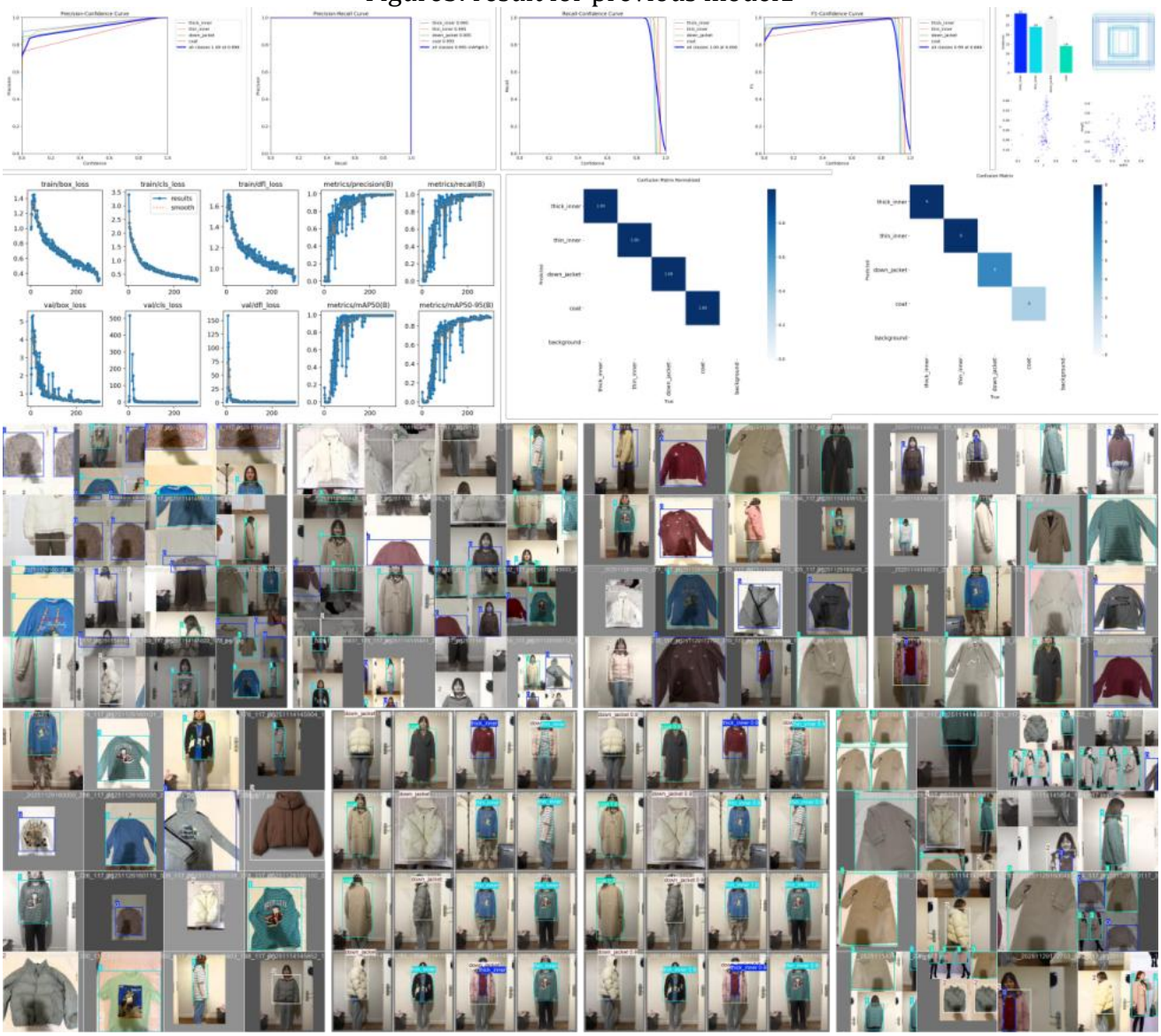


Figure4: result for previous model2